Ders 11 Genişletilmiş Notu: En Sıcak/En Soğuk Gün, Tekrar Eden Eleman, Mantıksal VE/VEYA ve Karakter Dizileri

Ders 11 Genişletilmiş Notu: En Sıcak/En Soğuk Gün, Tekrar Eden Eleman, Mantıksal VE/VEYA ve Karakter Dizileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. İsteneni Çözmek ve Uç Durumları Düşünmek
- 2. Akış Diyagramında Üs Alma ve Fonksiyon Kullanımı
- 3. En Sıcak ve En Soğuk Gün Problemi
- 4. Dizide Tekrar Eden Eleman Problemi
- 5. Mantıksal VE ve VEYA
- 6. While ile Çözümün Tamamı ve Analizi
- 7. Karakter Dizileri ve ASCII Karşılıkları
- 8. Alıştırma: Karakter Dizisini Tam Sayıya Çevirmek

1. İsteneni Çözmek ve Uç Durumları Düşünmek

Derse, bir problem çözülürken istenenin dışına çıkmanın ne anlama geldiği konuşularak başlandı.

Soru: Bir çözüme, problemin istemediği ek bir iş eklemek (örneğin dizide yer değiştirme yapan bir bölüm) sorun olur mu?

Cevap: Hoca bunu müşteri benzetmesiyle açıkladı. Müşteri sizden bir şey istediğinde, genelde istediğinin teslim edilmesini bekler. Daha fazlasını teslim ederseniz memnun olur. Ama alakasız bir şey teslim etmek, müşterinin aklında bir tereddüt oluşturur: Karşı taraf benim söylediklerimi mi yapıyor, yoksa kendi bildiğini mi? Bu nedenle çözüm istenenler çerçevesinde kalmalıdır. Yine de o problemle ilgili, kendi katkınız olan anlamlı bir ekleme olumlu karşılanır.

Bunun iyi bir örneği uç durumlardır. Hoca, bir problemde konum değerinin eksi olabilme ihtimalini düşünmeyi örnek verdi. Problemde böyle bir durum özellikle belirtilmemiş olsa da, mümkünse ek durumları (extra case) dikkate alarak çözmek sizi geliştiren bir yaklaşımdır ve hoca bunu destekliyor. Zorunlu olan ise temel durumların, burada sıfır ve pozitif değerlerin, doğru çözülmesidir.

2. Akış Diyagramında Üs Alma ve Fonksiyon Kullanımı

Üstel gösterim (2^5 gibi) akış diyagramında yazılabilir görünse de normalde geçerli (legal) bir gösterim değildir. Bunun yerine, üs alma işlemi for döngüsünde çarpma tekrarlanarak yapılır. Hocanın örneği 2^5 idi: 2 sayısı 5 defa birbiriyle çarpılır.

$$2^5 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 32$$

(Sonuç değeri olan 32 ara adım olarak eklendi.)

Hoca bu noktada daha önce söylediği bir ilkeyi hatırlattı: Dört işlem, for, while ve if, aslında bütün problemlerin çözümünü oluşturur. Fonksiyonlar ve metotlar bunların özelleştirilmiş hâlleridir. İleride bunları fazlasıyla kullanacaksınız, geri kalan hayatınızda üs almayı fonksiyon kullanmadan yapmayacaksınız. Ama şu anda temel öğreniliyor, bu yüzden bu tür durumlarda fonksiyon kullanılması istenmiyor.

Bunun birkaç istisnası vardır. Örneğin problem karekök almayı gerektiriyorsa `sqrt` fonksiyonu çağrılır, çünkü bunun akış diyagramında bir karşılığı yoktur. Hocaya göre Bilgisayar Bilimlerine Giriş dersinin sınırları içinde bu tür birkaç istisna bulunur. Sınırlar genişledikçe fonksiyon kullanımına yavaş yavaş daha serbest biçimde geçilecektir.

3. En Sıcak ve En Soğuk Gün Problemi

Problem şöyledir: Bir şehrin 365 günlük sıcaklık bilgisi veriliyor. Yılın en sıcak ve en soğuk günlerini bulup sıcaklık değerleriyle birlikte ekrana yazdıran algoritmanın akış diyagramı çizilecektir. Çözüm, öğrencilere adım adım sorularak kuruldu.

3.1 Veriyi Okumak

Akış `start` ile başlar. Kullanıcıya 365 günün sıcaklık değerlerini girmesini söyleyen bir mesaj verilir. Sonra birden başlayıp birer artarak 365'e kadar (365 dahil) giden bir for döngüsü açılır. Döngünün her adımında girilen değer `T` isimli dizinin `I`'ıncı elemanına okunur. Böylece 365 günün değerleri bir diziye alınmış olur.

3.2 Başlangıç Değerleri ve Değişken İsimleri

En büyük ve en küçük sıcaklık için iki değişken gerekir. İkisine de başlangıç değeri olarak dizinin ilk elemanı `T[1]` atanır. Ama sadece sıcaklığı bilmek yetmez, hangi gün olduğu da istenmektedir. Bu yüzden iki değişken daha gerekir ve ilk eleman başlangıç alındığı için ikisinin değeri de 1 olur.

Burada değişken isimleri üzerine uzun bir konuşma yapıldı. Bir öğrenci bu değişkenlere `count1` ve `count2` demişti. Hoca, `count1`'in kendisine hiçbir şey ifade etmediğini söyledi. `en büyük gün` önerisini de tam uygun bulmadı, çünkü asıl aradığımız en sıcak gündür. Sonunda isimler `enSicakGun` ve `enSogukGun` oldu.

Hocanın gerekçesi şudur: `count1` ve `count2` yalnızca yazan kişi için bir şey ifade eder. Bu tür kısa programlarda sıkıntı çıkarmaz, ama algoritma uzadığında akış diyagramını takip eden biri için yalnızca bir şey saydığınızı anlatır. Başka bir isim vermemek hata sayılmaz. Yine de hoca, değişken isimlerinin algoritma içinde bir şeyi temsil etmesine bugünden alışılmasını istedi. Aynı gözle `en büyük` ve `en küçük` isimlerinin de aslında çok doğru olmadığını belirtti.

3.3 Karşılaştırma Döngüsü

Birinci gün zaten hem en sıcak hem en soğuk kabul edildiği için ikinci for döngüsü 2'den başlar, birer artarak 365'e kadar gider.

Döngünün içinde önce `T[I] >= enbuyuk` kontrol edilir. Doğruysa en büyük değer güncellenir: `enbuyuk = T[I]`.

Sonra gün bilgisi de güncellenmelidir. Bir öğrenci burada `enSicakGun = enSicakGun + 1` yazmayı denedi. Hoca bunun neden yanlış olduğunu bir soruyla gösterdi: Saklamak istediğimiz şey nedir? Saklanan, en sıcak günün kaçıncı gün olduğudur. En büyük sıcaklık değeri değiştiyse gün de değişmiştir ve yeni gün, o an bakılan `I`'ıncı gündür. O hâlde doğru atama `enSicakGun = I` olur. Burada bir şey toplanmaz, artırılmaz, doğrudan atanır.

3.4 En Büyük ve En Küçük Tek Döngüde

Bir öğrenci, en sıcak günü bulduktan sonra True ve False kollarını birleştirip döngüyü sonlandırmayı ve en küçüğü ayrı bir döngüde aramayı önerdi. Hoca bu fikrin birçok kişide olduğunu belirterek şu yorumu yaptı: Birinci döngüde en büyüğü, ikinci döngüde en küçüğü bulursanız diziyi iki kez gezmiş olursunuz (hocanın ifadesiyle iki defa 365 kere). İşleri aynı döngü içinde

bitirme şansımız varsa yeni bir döngü açmak istemeyiz, çünkü yeni bir döngü daha fazla kontrol ve aynı turu yeniden atmak demektir.

> **Hocanın vurgusu:** İlk döngüde en büyüğü, ikinci döngüde en küçüğü bulmak yanlış değildir. Ama bunun iyileştirilmiş hâli, en büyük ve en küçük değeri aynı döngüde bulmaktır. Mümkünse yeni bir döngü açılmaz.

Peki aynı döngüde nasıl? Hoca sorarak ilerledi. `T[I]` en büyükten büyük ya da eşit değilse, en büyükten küçüktür. Eşitlik zaten koşula dahil edildiği için geriye eşit olma ihtimali kalmaz. Bir öğrenci bu durumda o günün en soğuk gün olduğu sonucuna vardı. Hoca bunu düzeltti: En büyükten küçük olması, o günün en soğuk gün olduğunu göstermez. Yalnızca en soğuk gün olma ihtimalini doğurur, bunu ayrıca kontrol etmek gerekir.

Bir başka öğrenci koşulu `T[I] >= enkucuk` biçiminde kurup atamaları onun False koluna koymayı önerdi. Hoca, `if` kutusunun içine böyle bir ifade yazmak istemediğini söyleyip başka bir öğrenciye geçti. Kabul edilen çözüm şu oldu: İlk `if`'in False kolunda yeni bir `if` açılır ve `T[I] <= enkucuk` kontrol edilir. Doğruysa `enkucuk = T[I]` ve `enSogukGun = I` atanır. False kolunda yapılacak bir şey yoktur, kollar birleştirilip döngüye geri dönülür. En üstteki `if`'in False kolunda da bu ikinci kontrol dışında bir iş yoktur.

3.5 Sonucu Yazdırmak

Döngüden çıkıldığında aranan her şey bulunmuştur, geriye yalnızca yazdırmak kalır. Hoca, algoritmanın sonunda ne yapılacağını öğrencilere sorarak bunu vurguladı: Çıktı kutusuna tırnak içinde `En sıcak gün=` mesajı ile `enSicakGun` değişkeni, tırnak içinde `En soğuk gün=` mesajı ile `enSogukGun` değişkeni yazılır. Problem sıcaklık değerlerini de istediği için `enbuyuk` ve `enkucuk` da kutuya eklenir. Algoritma `stop` ile biter.

Algoritmanın tamamı numaralı adımlarla şöyledir:

1. `start`.
2. Mesaj: 365 günün sıcaklık değerlerini giriniz.
3. `I`, 1'den 365'e kadar birer artarak: `T[I]` değerini oku.
4. `enbuyuk = T[1]`, `enkucuk = T[1]`, `enSicakGun = 1`, `enSogukGun = 1`.
5. `I`, 2'den 365'e kadar birer artarak:
   - `T[I] >= enbuyuk` doğruysa: `enbuyuk = T[I]`, `enSicakGun = I`.
   - Yanlışsa `T[I] <= enkucuk` kontrol edilir:
     - Doğruysa: `enkucuk = T[I]`, `enSogukGun = I`.
     - Yanlışsa: bir şey yapılmaz.
6. Çıktı: `'En sıcak gün='`, `enSicakGun`, `enbuyuk`, `'En soğuk gün='`, `enSogukGun`, `enkucuk`.
7. `stop`.

3.6 Eşitlik ve Aynı Sıcaklıktaki Günler

Soru: >= ve <= içindeki eşitlik, en büyük ya da en küçük değeri değiştirmiyor, yalnızca günü değiştiriyor. İki gün aynı değere sahipse ne olur?

Cevap: 365 gün içinde aynı sıcaklığa sahip günler doğal olarak olabilir. Bu algoritma çalışmaya devam eder ve aynı en yüksek sıcaklığa sahip günlerden en sonra geleni gösterir. Aynı şekilde en düşük sıcaklık için de en sonra gelen günü gösterir. Diğer günleri göstermez, ama bu yanlış sayılmaz. Problemde bütün en sıcak ve en soğuk günler istenseydi tek bir gün saklamak yetmezdi. Bu günleri bir en sıcak gün dizisinde ve bir en soğuk gün dizisinde saklayıp birlikte yazdırmak gerekirdi. Birçok problemde en büyük ya da en küçük elemanın birden fazla olabileceği görülecektir.

Soru: Dizileri kullansaydık bu döngünün dışında bir döngüye daha ihtiyaç olur muydu?

Cevap: Evet, önce en büyük bulunur, sonra bu diziler farklı bir döngüde doldurulur. Hoca bunun başlı başına başka bir algoritma olduğunu söyledi ve ayrıntısına girmedi. İsteyen bunu kendisi için alıştırma olarak çözebilir. Hoca, ayrıntısı anlatılmadan bazı kişilerin anlayıp bazılarının anlayamayacağı yarım yamalak bir öğrenmeyi tercih etmediğini belirtti.

3.7 Sıralı Veri Olsaydı

Soru: Sıcaklıklar sıralı verilseydi, bu diyagramda en küçük kontrolüne hiç geçilmeme ihtimali oluşuyor, değil mi?

Cevap: 365 günün sıcaklıklarının birinci günden başlayarak soğuktan sıcağa sıralı gitme ihtimali zaten yoktur. Ama başka bir veri setinde veri size sıralı verilirse, en küçük ve en büyüğü bulmak için dizinin tamamını gezmeniz gerekmez. İlk elemanı en küçük, son elemanı en büyük olarak alırsınız. Dizi büyükten küçüğe sıralıysa bunun tersi geçerlidir.

> **Ders sonrası not:** Hoca büyükten küçüğe sıralı durumu anlatırken "ilk eleman en büyük, ikinci eleman en küçük" dedi; doğrusu ilk eleman en büyük, son eleman en küçüktür. Az önceki cümlesinde kendisi de son elemandan söz ediyordu.

3.8 İkinci Kontrol Neden False Kolunda?

Soru: İlk `if`'in True ve False kollarını birleştirip en küçük kontrolünü ondan sonra yeni bir `if` ile yapsak olmaz mı?

Cevap: Hoca soruyu sınıfa yöneltti ve bir öğrencinin verdiği cevabı onayladı: Bir değer en büyükten büyükse en küçükten küçük olamaz. O hâlde en büyükten büyük olduğu yerden çıktıktan sonra bir daha "en küçükten küçük mü?" diye sormak doğru olmaz. Bu kontrol bu yüzden False kolunda yapılır. Arka arkaya iki `if` koysaydınız, bu soruda biraz daha fazla işlem yapmış olurdunuz.

> **Hocanın vurgusu:** Bu tür en büyük/en küçük problemlerinde, değer en büyükten büyük değilse en küçükten küçük olma ihtimali vardır. En büyükten büyükse bu ihtimal yoktur, dolayısıyla ikinci kez sormak doğru olmaz.

Soru: Eşitlik yüzünden sorun çıkmaz mı? Örneğin ilk gün 1, ikinci gün de 1 olsa değer en büyük dalına girer, en küçük dalına girmez, bu yüzden en küçüğün günü tutulmaz.

Cevap: Hoca örneği izledi. İlk gün 1, ikinci gün 1 olsun. Başlangıçta `enbuyuk = 1`, `enkucuk = 1`, `enSicakGun = 1`, `enSogukGun = 1` olur. İkinci günde $1 \geq 1$ doğru olduğu için `enSicakGun = 2` olur, False koluna girilmediği için `enSogukGun = 1` kalır. Sonuçta en sıcak gün ikinci gün, en soğuk gün birinci gün görünür. Hoca iki günün değeri aynı olduğu için birinci ya da ikinci gün olmasının çok fark etmediğini, bunun sıkıntı yaratmayacağını, ancak çok özel bir durum için belki sıkıntı yaratabileceğini söyledi.

> **Ders sonrası not:** Hoca 3.6'daki soruda en düşük sıcaklık için de en sonra gelen günün gösterileceğini söyledi; bu genel kural öğrencinin bu örneğinde tutmuyor. Birinci ve ikinci gün aynı en düşük değere sahipken sonuç, en sonra gelen ikinci gün değil birinci gündür. Doğrusu şudur: Değer `>=` koşulunu sağladığında en küçük kontrolüne hiç bakılmaz, bu yüzden en küçüğe eşit bir değer en büyüğe de eşitse `enSogukGun` güncellenmez. Bulunan sıcaklık değerleri ve hocanın vardığı "aynı değerlere sahipler" sonucu değişmez.

4. Dizide Tekrar Eden Eleman Problemi

Problem şöyledir: Verilen bir dizide tekrar eden eleman olup olmadığını bulan algoritmanın akış diyagramı çizilecektir.

4.1 Problemi Örneklerle Anlamak

Hoca önce sorunun ne dediğini öğrencilere sordu. Bir öğrencinin tekrar eden elemanları bulup dizide işaretlemek olarak yorumlaması doğru bulunmadı. İstenen, tekrar eden elemanların kendisi değil, tekrar olup olmadığı bilgisidir: Tekrar varsa "tekrar vardır", yoksa "yoktur" denmesi. Hoca bir örnek vermenin zararı olmadığını, hep örnekler üzerinden anlatıldığını belirterek şu dizileri kullandı:

- Tekrar edenler: `1, 3, 3, 5`, `1, 3, 5, 3`, `1, 1, 2`, `2, 2, 1, 4`.
- Tekrar etmeyenler: `6, 2, 1, 3`, `3, 5, 0, 1`.

Tekrar etmeyen diziye elemanları unique olan dizi denir. Amaç, elemanlar incelendiğinde tekrar eden eleman varsa ekrana dizinin unique olmadığını, yoksa unique olduğunu yazan bir algoritma tasarlamaktır.

Soru: Bir tane bile tekrar eden eleman bulsak durdurmamız yeter mi?

Cevap: Aynen öyle. Daha fazla sayıda tekrar eden eleman ya da eleman grubu olabilir, ama bir tanesinin koşulu bozması yeterlidir.

Soru: Dizinin eleman sayısı belli olacak mı, yoksa kullanıcıdan mı alınacak?

Cevap: Her zaman olduğu gibi, dizinin elemanlarını alabilmek için önce eleman sayısını almak gerekir. Buradaki örnekler yalnızca 4 elemanlıdır, problem N elemanlı dizi için çözülecektir.

4.2 Öğrencilerin Denemeleri ve İzin Verilmeyen Çıkışlar

Öğrencilere çözmeleri için süre verildi, sonra denemeleri konuşuldu. Üç for döngüsü ve çok sayıda `if` kullanıp çözümünün uzun olduğunu düşünen de vardı, for ile çözüp bulduktan sonra da çalışmaya devam ettiğini fark eden de.

Soru: For döngüsünün içindeki `if`'ten sonra `stop` koydum, bu mümkün mü?

Cevap: Maalesef mümkün değil. For döngüsünün içinde bir `if` varsa ve `if`'in sonuna `stop` koyduysanız bu yanlıştır.

Soru: Python'da while'da `break` diye bir şey var, burada for'u bitirmek için kullanılamaz mı?

Cevap: Hocaya göre `break` kullanımı doğru bir kullanım değildir, çünkü kodun akışını illegal olarak bozar. While içinde `break`'e ihtiyaç duyuyorsanız, while koşulunu ona göre belirlemeniz gerekir.

> **Ek bilgi (derste anlatılmadı):** `break`, C ve Python'da dilin geçerli bir deyimidir, program bu deyimle hata vermez. Hocanın kuralı büyük olasılıkla bir söz dizimi kuralı değil, bu derste öğretilmek istenen tasarım tercihidir: Döngünün ne zaman biteceği döngü koşuluna bakılarak anlaşılabilmelidir.

Soru: Tekrar eden sayı bulunca, döngüyü hemen bitirmek için for'un indisini `if`'ten sonra son değere eşitledim. Olur mu?

Cevap: Bu da doğru değildir. N'e kadar giden bir döngüde belirli bir adımda kontrol yapıp `I`'yı N'e eşitleyerek çıkılmamalıdır.

> **Hocanın vurgusu:** Ne `break` ile ne de indisi değiştirerek for döngüsünden çıkılmalıdır.

Bir öğrenci de bir for döngüsünde `A1 = A2`, `A2 = A3` gibi, N'e kadar giden karşılaştırmalar yaptığını anlattı. Hoca bunun üzerine "for mu, while mı kullanacağız?" sorusunu sordu. Öğrencilerin bir kısmı, eşitlik bulununca hemen çıkabilmek için elemanları bir for ile okuyup sonra iki while kullanmayı, bir kısmı ise bir while ve bir for kullanmayı önerdi.

> **Ek bilgi (derste anlatılmadı):** Yalnızca yan yana duran elemanları karşılaştırmak bu problemde büyük olasılıkla yetmez, çünkü hocanın belirttiği gibi veri sıralı değildir. Derste geçen `1, 3, 5, 3` dizisinde tekrar eden iki 3 yan yana değildir, bu yüzden böyle bir karşılaştırma tekrarı görmez.

4.3 Veriyi Okumak ve Kağıt Üzerinde Düşünmek

Çözümün başı önceki problemlere benzer: `start`, eleman sayısını isteyen mesaj, N'in okunması, elemanları isteyen mesaj ve `I`, 1'den N'e kadar giden bir for döngüsüyle `Dataset[I]` değerlerinin tek tek okunması. Veri karışık verilmiştir, sıralı değildir. Sıralı olsaydı daha farklı bir algoritma uygulanabilirdi.

Hoca akış diyagramından önce şunu sordu: Bu işi kağıt üzerinde yapsaydınız, sözel olarak nasıl organize ederdiniz? Cevap şudur: Birinci eleman kendisinden sonraki tüm elemanlarla karşılaştırılır. Sonra birinci eleman bir kenara bırakılır, ikinci eleman kendisinden sonrakilerle karşılaştırılır. Sonra üçüncü eleman ve sonrakiler. Birinci elemanla karşılaştırması zaten yapıldığı için her turda bakılacak eleman sayısı bir azalır.

4.4 İç İçe For ile Çözüm

Bu fikir önce iç içe for döngüleriyle kuruldu:

- **Dış döngü:** `I`, 1'den başlar ve $N-1$'e kadar gider. N'e kadar gitmez, çünkü en sonda karşılaştırılacak bir eleman payı bırakılmalıdır.
- **İç döngü:** `J`, `I + 1`'den başlar ve N'e kadar gider. Dış döngüde birinci eleman alındıysa iç döngüde ikinciden, ikinci eleman alındıysa üçüncüden başlanır. Örneğin `1, 3, 5, 3` dizisinde en sonda dış döngüde 5 alındığında, iç döngüde geriye tek bir 3 kalır.
- **Kontrol:** `Dataset[I] = Dataset[J]` ise tekrar bulunmuştur.

Tekrarın sonuca nasıl yansıtılacağı için bir öğrenci iki yol saydı: Bir değişken tutup artırmak ve sonda sıfırdan farklı olup olmadığına bakmak ya da her seferinde mesaj bastırmak. Hoca çözümde bir `unique` değişkeni kullandı: `unique` başta 1 yapılır, tekrar bulunursa 0 yapılır. Döngüler bittikten sonra `unique` 1 ise True kolunda `Unique`, değilse False kolunda `Not unique` yazılır.

Bu çözümün neden iyi olmadığı da soruldu. İki for döngüsünün de sonuna kadar gidilmektedir, oysa ilk eşitlik bulunduğunda zaten çıkılmalıdır. Hoca, for yapısında `break` ve benzeri çıkış noktalarıyla ya da `J`'yi N'e, `I`'yı $N-1$'e eşitleyerek döngüyü dış güçle bitirmenin istenmeyen bir kullanım olduğunu yineledi. Burada adım sayısı belli olmayan bir problem vardır: Verilen girdiye göre birinci adımda da çıkılabilir, sonuna kadar da gidilebilir. Bu nedenle hoca çözümün yanlış olduğunu söylemedi, ama üzerine kocaman bir ünlem işareti koyarak tercih edilmemesi gerektiğini belirtti.

Soru: Kaç kez tekrar ettiğini bulmak isteseydik bu çözümü kullanabilir miydik?

Cevap: Evet. Her elemanın kaç kez tekrar ettiği isteseydi, sonuna kadar gidildiği için bu yapı kullanılabilirdi.

5. Mantıksal VE ve VEYA

Tasarım while ile baştan yapılırken yeni bir kavram ortaya çıktı. Önce `unique = 1` denilerek dizinin unique olduğu varsayılır, sonra `I = 1` yapılır. Dış while döngüsünün koşulu öğrencilere soruldu. Cevap iki parçalıydı: `I < N` ve `unique = 1`. Bu iki koşul neyle bağlanacaktır? Öğrenci C'deki gösterimi söyledi, hoca da bunun adının "and" olduğunu belirtti.

Hoca bu kullanımın ilk defa konuşulduğunu belirterek şöyle tanımladı:

- **Mantıksal VE (logical AND):** Döngüde kalmak için iki koşulun birlikte sağlanması gerekiyorsa koşulların arasına "ve" yazılır. `I < N` olduğu sürece ve `unique` 1 olduğu sürece döngü devam eder.
- **Mantıksal VEYA (logical OR):** İki koşuldan birinin sağlanması döngünün devam etmesine yeterliyse "ve" yerine "veya" yazılır.

Bu kullanım yeri geldiğinde `if`'te de görülür, ama ağırlıklı olarak while döngüleri içindir ve çok sık kullanılacaktır. Bir döngüde birden fazla "ve", bir "ve" ile bir "veya" gibi birleşimler de kullanılabilir. Şu an için problemler genelde tek bir "ve" ya da "veya" ile çözülüyor, ama ilerleyen zamanlarda bir döngünün devam etmesi için ikiden fazla şartın sağlanması da gerekebilir.

> **Ek bilgi (derste anlatılmadı):** C dilinde mantıksal VE büyük olasılıkla öğrencinin kastettiği &&, mantıksal VEYA ise || işlecidir. Bu iki gösterim dersin kısa notunda da yer alıyor, altyazıda ise açıkça okunamıyor.

6. While ile Çözümün Tamamı ve Analizi

6.1 Akışın Kurulması

Dış döngüye girildikten sonra iç döngünün başlangıcı ayarlanır: `J = I + 1`. Bir sonraki elemandan başlamak gerekir. Bir öğrenci önce `I - 1` deyip hemen düzeltti.

Sonra iç while açılır. Koşulun ilk parçası `J <= N`'dir, küçük değil küçük eşit, çünkü `J` son elemana kadar gitmelidir. Ama bu yetmez: Bir koşul bozulduğunda bu döngüden de çıkılmalıdır. Bu yüzden iç döngüye de `unique = 1` eklenir: `J <= N AND unique = 1`.

İç döngünün içine bir `if` kutusu gelir: `Dataset[I] = Dataset[J]`. True kolunda iç döngüden çıkılmak istenmektedir. Bunun için `break` değil, koşulu bozan atama kullanılır: `unique = 0`. False kolunda bir şey yapılmaz, kollar birleştirilir.

> **Hocanın vurgusu:** "Çok önemli bunlar." While'da döngü indislerini sizin artırmanız gerekir. For'da indis otomatik olarak artar, bunu unutmayın.

Bu nedenle `if`'ten sonra iç döngüde `J = J + 1` yazılır. İç döngüden çıkınca dış döngüde `I = I + 1` yazılır. Her iki döngüden de çıkıldıktan sonra `unique` kontrol edilir: 1 ise `Unique`, değilse `Not unique` yazılır ve `stop` ile bitilir.

```
start
mesaj: eleman sayısını giriniz
N oku
mesaj: elemanları tek tek giriniz
for I = 1 ... N
    Dataset[I] oku
unique = 1
I = 1
while (I < N AND unique = 1)
    J = I + 1
    while (J <= N AND unique = 1)
        if Dataset[I] = Dataset[J]
            True:  unique = 0
            False: (işlem yok)
        J = J + 1
    I = I + 1
if unique = 1
    True:  'Unique' yaz
    False: 'Not unique' yaz
stop
```

Soru: Unique bozulduğunda ilk while'dan çıkılmış olmayacak mı? İkinci while'ı da unique'e bağlamaya gerek var mı?

Cevap: Hoca çözüm tamamlandıktan sonra soruyu yeniden sordu. Soran öğrenci çözümü görünce ikinci while'dan da çıkılması gerektiğini anladığını söyledi.

Soru: `unique`'e 1 demek yerine tırnak içinde bir metin atasak, sonra değiştirip böylece alttaki `if`'i kullanmasak olmaz mıydı?

Cevap: Olmaz. Metin yazsanız da yazmasanız da değeri kontrol etmeniz gerekir, `if` kullanmak zorundasınız. Bu tür fikirler, ileride programlama dillerinin verdiği bazı ek yeteneklerle bir iki kısaltma yapmaya dayanır. Şu anki problem çözme biçimimiz değildir ve akış diyagramında bu tip kullanımlar yapılmaz.

6.2 Analiz: Tekrarsız Dizi `1, 3, 5, 7`

Analizde izlenecek değişkenler öğrencilere soruldu: `N`, `I`, `J`, `Dataset[I]`, `Dataset[J]` ve `unique`. Verilen dizi `1, 3, 5, 7`, yani $N = 4$. Başlangıçta `I = 1`, `unique = 1`.

`I`	`J`	`Dataset[I]`	`Dataset[J]`	Eşit mi?	Adımdan sonra `unique`
1	2	1	3	Hayır	1
1	3	1	5	Hayır	1
1	4	1	7	Hayır	1
2	3	3	5	Hayır	1
2	4	3	7	Hayır	1
3	4	5	7	Hayır	1

Döngü koşullarıyla birlikte akış şöyledir:

1. `I = 1`: $1 < 4$ ve `unique = 1` olduğu için dış döngüye girilir, `J = 2` olur. `J` sırasıyla 2, 3, 4 değerlerini alır, üç karşılaştırmada da eşitlik yoktur. `J = 5` olunca $5 \leq 4$ sağlanmaz, iç döngüden çıkılır, `I = 2` olur.
2. `I = 2`: $2 < 4$ ve `unique = 1`, `J = 3` olur. `J` 3 ve 4 değerlerini alır, eşitlik yoktur. `J = 5` olunca çıkılır, `I = 3` olur.
3. `I = 3`: $3 < 4$ ve `unique = 1`, `J = 4` olur. 5 ile 7 eşit değildir. `J = 5` olunca çıkılır, `I = 4` olur.
4. `I = 4`: $4 < 4$ sağlanmaz, dış döngüden çıkılır.
5. `unique` hâlâ 1 olduğu için ekrana `Unique` yazılır.

> **Ders sonrası not:** Hoca analizi okurken iç döngü için kimi yerde "4 küçük 4 mü?" ve "J küçük N" dedi; koşul küçük eşittir, doğrusu "$4 \leq 4$ mü?" ve $J \leq N$'dir. Kendisi de izlemenin sonuna doğru "küçük eşit" diye düzeltti.

6.3 Analiz: Tekrarlı Dizi `1, 1, 3, 5`

Verilen dizi `1, 1, 3, 5` olsaydı:

1. `I = 1`, `unique = 1`, dış döngüye girilir, `J = 2` olur.
2. `Dataset[1] = 1` ve `Dataset[2] = 1` eşit olduğu için `unique = 0` olur.
3. `J` 3 olur (ara adım eklendi). `unique` 0 olduğu için iç döngüden ilk turda çıkılır.
4. `I` 2 olur. Dış döngüde `unique` 1'e eşit olmadığı için oradan da çıkılır.
5. Son kontrolde `unique` 1 olmadığı için ekrana `Not unique` yazılır.

Soru: `unique` 0 olunca döngüden nasıl çıkıldığını göremedim.

Cevap: İç döngü `J <= N` ve `unique = 1` olduğu sürece dönüyor. Birinci koşul `J <= N`, ikinci koşul `unique = 1`'dir. Bu ikinci koşul `if`'in True kolundaki atamayla kırılır.

6.4 Kaba Kuvvet ve Karmaşıklık

Böylece for kullanmadan, `break` kullanmadan ve for indisine dışarıdan müdahale etmeden sorunun nasıl çözüldüğü görüldü.

> **Hocanın vurgusu:** Adım sayısı belli olmuyorsa `break` kullanılmaz, while kullanılır.

Burada iç içe iki döngü yapıldığı için aslında biraz fazla iş yapılmaktadır. Hocaya göre bunun ilerleyen senelerde görülecek daha optimal, daha verimli çözümleri vardır. İlk akla gelen bu çözüme kaba kuvvet (brute force) çözüm denir. İç içe iki döngü yapıldığında karmaşıklık otomatik olarak bir anda N^2'ye çıkar. Hoca, yüz eleman olsaydı oldukça yüksek sayıda karşılaştırma yapmak zorunda kalınacağını belirtti.

> **Ek bilgi (derste anlatılmadı):** Tekrar bulunmazsa karşılaştırma sayısı $(N-1) + (N-2) + \cdots + 1 = \frac{N(N-1)}{2}$ olur. Derste izlenen $N = 4$ durumunda bu, tablodaki 6 satırdır. Hocanın andığı yüz elemanda sayı büyük olasılıkla 4950 karşılaştırmaya ulaşır, N^2 denmesinin nedeni de büyümenin bu ifadedeki N^2 teriminden gelmesidir.

7. Karakter Dizileri ve ASCII Karşılıkları

Dersin sonunda, perşembe günü ayrıntısına girilecek konuya kısa bir giriş yapıldı. Değişken tipleri arasında tam sayı ve kesirli sayının yanında karakter saklayan değişkenler de vardı. Doğal olarak karakter dizileri de vardır.

Bir tam sayı dizisi (array of integers) düşünelim: `1, 2, 3, 4`. Bir de `A, B, C` gibi karakterleri sakladığımız diziler vardır. Bu dizilerde karakterlerin kendisi değil, sayısal karşılıkları saklanır. Sayısal karşılık için ASCII tablosuna bakılır.

Harfler böyle saklandığına göre, rakam karakterleri saklandığında da aynısı geçerlidir. Aşağıdaki karşılaştırma aradaki farkı gösterir:

Dizi	Elemanlar	Bellekte saklanan değerler
Tam sayı dizisi	`1, 2, 3, 4`	1, 2, 3, 4

Dizi	Elemanlar	Bellekte saklanan değerler
Karakter dizisi	`'1', '2', '3', '4'`	49, 50, 51, 52

Hoca bu değerleri, yıllardır anlattığı için ezberinden yazdığını söyledi. Bir kural vardır: `'0'` ile `'9'` arasındaki rakam karakterleri, küçük `a` ile küçük `z` arasındaki harfler ve büyük `A` ile büyük `Z` arasındaki harfler tabloda art arda gelir. Ama kaçtan başladıklarını bilmek zorunda değilsiniz.

> **Hocanın vurgusu:** ASCII tablosu ezberlenmeyecek ve sayısal değerler hiçbir zaman, en azından bu ders boyunca, kullanılmayacak.

Art arda gelme kuralı hesap yapmak için yeterlidir. Karakter sabitlerinin farkı sayısal bir sonuç verir:

- `'1' - '0'` sayısal olarak 1'dir.
- `'9' - '0'` sayısal olarak 9'dur.

8. Alıştırma: Karakter Dizisini Tam Sayıya Çevirmek

Dersin alıştırması 11. algoritmadır: N rakamdan oluşan bir karakter dizisindeki değeri tek bir tam sayı değerine dönüştüren algoritmanın akış diyagramı çizilecektir. Hoca örneklerle anlattı:

- `'1', '2', '3', '4'` dizisinin tam sayı karşılığı 1234'tür.
- `'3', '0', '5'` dizisinin tam sayı karşılığı 305'tir.
- Tek rakamlı `'1'` dizisinin karşılığı, başka bir tam sayı değişkenine 1 olarak yazılacaktır. Oysa bu karakterin bellekteki değeri 49'dur.

Soru: Dizi sıfırla başlarsa, örneğin sıfır, üç, beş verilirse ne olur?

Cevap: Şu an için dikkate alınmayacak, dizi sıfırla başlamayacaktır. Hoca bunun güzel bir uç nokta olduğunu, algoritma geliştirme yeteneği ilerledikten sonra buna bakılacağını söyledi.

Soru: ASCII tablosundaki değerleri kullanabilir miyiz?

Cevap: Kullanılmayacak. Kullanılması gereken yalnızca karakter sabitleridir. `'1'` karakter sabitinden `'0'` karakter sabiti çıkarıldığında sayısal olarak 1 kalır, `'9'`'dan `'0'` çıkarıldığında sayısal olarak 9 kalır. Problem bu bilgi kullanılarak çözülecektir.

Soru: Basamak basamak ilerleyerek, üs almadakine benzer bir yöntem mi uygulanacak?

Cevap: Hoca çözüm kısmını söylemedi, yalnızca çözüme yönelik ipuçları verdiğini belirtti. Analiz yapıldığında çözümün doğru olup olmadığı zaten görülecektir. Çözüm bir sonraki derste konuşulacaktır.

Dersin Toparlaması

Bu derste bir dizi üzerinde iki temel karar verildi. En sıcak ve en soğuk gün probleminde, adım sayısı belli olduğu için for kullanıldı. Başlangıç değerleri ilk elemandan alındı, karşılaştırma 2'den başladı ve en küçük kontrolü, en büyük kontrolünün False koluna kondu. Böylece hem gereksiz bir ikinci döngüden hem de gereksiz bir ikinci kontrolden kaçınıldı. Tekrar eden eleman probleminde ise adım sayısı girdiye bağlı olduğu için iç içe while kullanıldı. Çıkış, `break` ya da indis müdahalesiyle değil, mantıksal VE ile kurulan koşulu `unique = 0` atamasıyla bozarak sağlandı. Bu kaba kuvvet çözümün karmaşıklığı N^2'dir.

Dersin sonunda karakter dizilerine geçildi. Karakterler bellekte ASCII karşılıklarıyla tutulur, ama hesaplarda sayısal değerler yerine karakter sabitleri ve aralarındaki farklar kullanılır. Konuya perşembe günü daha ayrıntılı devam edilecek ve artık yavaş yavaş programlamaya da geçilecektir. Karakter dizisini tam sayıya çevirme alıştırması da o derste konuşulacaktır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.